RISC-V: ASIC-Design Team

Charlie Liu
Electrical and Computer Engineering
Rice University
Houston, United States
cl228@rice.edu

Kelly Chan
Electrical and Computer Engineering
Rice University
Houston, United States
kc162@rice.edu

***Abstract*—The reduced instruction set computer five (RISC-V) ecosystem is growing rapidly, but a gap remains in transitioning register-transfer level (RTL) designs to manufacturable chips from the previous semesters of the RISC-V Capstone. This project aims to bridge that gap by implementing a complete application-specific integrated circuit physical design flow. It will begin from a basic ELEC 527: VLSI Systems Design framework, and then define a custom process to achieve the final Graphic Data System II (GDSII) file while optimizing power, performance, and area (PPA). Our solution includes adapting previous flows to our RISC-V core using industry standard tools such as Design Compiler and Innovus to create our own automated flow, beginning with synthesizing the RTL all the way to generating a GDSII layout. By next semester, our design should meet performance, power, and area constraints, and also be ready for actual tapeout with minor modifications.**

***Index Terms*—RISC-V, physical design, RTL, GDSII**

I. Introduction

First, it is important to understand why reduced instruction set computer five (RISC-V) is important, and how it impacts the engineering industry and markets. According to Synopsys, "RISC-V is an open-source instruction set architecture used to develop custom processors for a variety of applications, from embedded designs to supercomputers." [1] Since its inception in 2010, it has quickly and exponentially become an integral part of computing. Its skyrocketing popularity and projected market growth can be attributed to a few key features, including but not limited to:

- "Its open-standard nature, which allows collaboration and innovation across the industry." [1]
- "Common ISA, which helps make software development easier since all processors could potentially use the same architecture. Designers can use the same base ISA, from simple embedded devices to the largest supercomputers, tailoring their device to the needs of the market. Compared to previous ISAs, RISC-V ISAs have unique features and can be customized based on their requirements." [1]
- "Availability of smaller, energy-efficient, and modular options." [1]
- "Security features, which are available through open-source reference designs, software composition analysis tools, and security extensions. In addition, its open-source nature means that the entire RISC-V architecture can be scrutinized closely in the public domain, eliminating back doors and hidden channels." [1]

Thus, our project aims to take advantage of these features. The SwitchMCU, a capstone four semesters in the making, utilizes a custom RISC-V 32-bit core that has the ability to execute C programs, as well as allow for future custom instructions and additional peripherals. It is programmed as a general-purpose biomedical application processor, supports a 5-stage pipeline, and uses Harvard architecture. Furthermore, it contains synchronous and asynchronous First-In, First-Out (FIFO) data structures for data transmission, Universal Asynchronous Receiver-Transmitter (UART), Serial Peripheral Interface (SPI), and General-Purpose Input/Output (GPIO) peripherals. More can be found on the official GitHub website. [2]

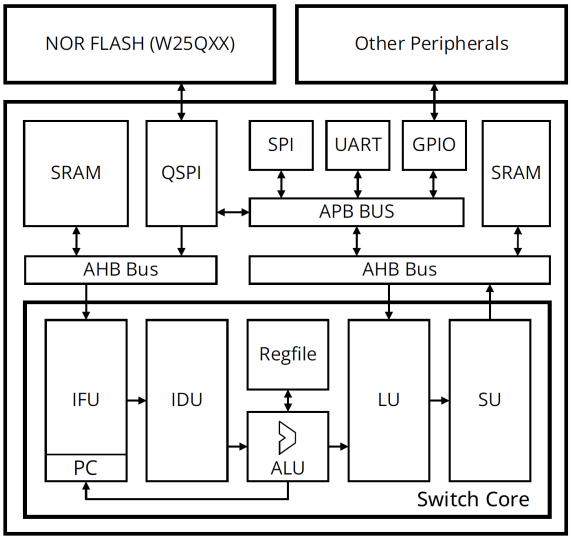

Fig. 1. System-Level Diagram of the RISC-V Based SwitchMCU.

The project can be broadly divided into two main sections: the core design and the physical design, which is the main focus of this paper. Physical design can be defined as "the process of transforming a circuit description into the physical layout, which describes the position of cells and routes for the interconnections between them." [3]

The core team designs, debugs, and verifies the RISC-V core, and will ultimately produce synthesizable RTL code to be passed along to the physical design team, which consists of the two authors. Once the Verilog code is received, it will be synthesized and ultimately taped out into a reproducible, manufactured chip.

As mentioned previously in the abstract, there is currently no standardized RISC-V Application-Specific Integrated Circuit (ASIC) workflow at Rice University. This is compounded by a more generalized problem in open-source physical design flows, which is confusing or poor documentation. This makes it difficult or even impossible for reproduction and restricts access to only those who are very familiar with the subject.

Therefore, the main assignments of this semester for the physical design team are providing fully documented instructions for each step of the physical design process, a GitHub repository (S25_pd) with all necessary TCL script files and code, and successfully synthesizing the RTL all the way to generating a GDSII layout. By next semester, the design should meet performance, power, and area (PPA) constraints, and also be ready for actual tapeout with minor modifications.

II. Methods

However, current work from the previous semesters requires revisions and additions before it is able to be transferred to a manufacturable chip. Therefore, a simple register file (`regfile.v`) from ELEC 526: High Performance Computer Architecture will be used as a substitute until the core team is able to complete the processor.

This file is then fed through a simplified physical design flow provided by ELEC 527: VLSI Systems Design Framework. These steps begin with a functional simulation in Questa. The design and testbench files, written in Verilog, are compiled, simulated, and run in the GUI. The waveforms are then analyzed to check for any potential issues.

Next, the output(s) are passed along to Synopsys Design Compiler for synthesis, which is the process of converting RTL into a gate-level netlist using standard cells. The design is optimized for performance, power, and area constraints. Then the synthesized Verilog output is once again examined in Questa.

The final component is place and route in Innovus. The software reads the netlist, places the standard cells within the chip layout, routes the interconnections between the cells, and generates both a physical design report and a GDSII layout. Since the AND structure files provided for this assignment have already been validated and refined, many intermediary steps between synthesis and place-and-route have been excluded.

Physical Design Flow

The full edited physical design flow for this project can be seen below in Fig.2:

1) **Synthesis** - Using Library Compiler with an input of `.lib` and an output of `.db`. A TCL script was created to convert all `.lib` files to `.db` files automatically due to the high volume of these found in the SkyWater 130nm PDK. The `.db` file is forwarded to Design Compiler, which outputs a `.vg` file.
2) **Floorplanning in Innovus** - This is "the process of creating core area, specifying core to I/O boundary spacing, standard cell rows, placing I/O pins, placing macros using macro guidelines, and adding placement blockages and halo." [4]
3) **Place and Route (P&R)** - Like floorplanning, place-and-route is achieved in Innovus. This is the physical implementation stage where logic cells are arranged and interconnected on a chip, and then streamlined. Since there were numerous `.lef` files in SkyWater with duplicates, a TCL script was written to handle them explicitly while avoiding redundant cases.
4) **Clock Tree Synthesis (CTS)** - The final step in Innovus is CTS, which "aims to minimize the routing resources used by the clock signal. Additionally, it minimizes the area occupied by the clock repeaters while meeting an acceptable clock skew, a reasonable clock latency, and clock transition time." [5]
5) **Parasitic Extraction (PEX)** - The `.gds` file is then given to StarRC for parasitic extraction, also known as Resistance-Capacitance (RC) extraction. This process involves determining both the resistance and capacitance values of interconnections and devices in a circuit layout to more accurately model signal delays and timing behavior.
6) **Static Timing Analysis (STA)** - The extracted file(s) are scrutinized in PrimeTime, during which the static timing analysis "provides full chip timing with signal integrity, advanced node accuracy, and ECO guidance." [6] As the name suggests, this stage is crucial for ensuring correct and reliable timing in the design.
7) **Design Verification** - The last part of the physical design flow is design verification using Calibre, which can be dissected into two parts: Design Rule Checks (DRC) and Layout Versus Schematic (LVS). DRC verifies that the physical layout of the chip adheres to the restrictions and requirements set by the manufacturer. Meanwhile, LVS ensures that the layout of the chip matches the netlist.

After all seven phases have been successfully completed, the design can be sent to the manufacturer for tapeout. Actual tapeout can span across a period of several months.

Results (Partial)

As of this report, the project is approximately 50% complete. The remaining tasks for the rest of the semester include Routing (Place and Route), RC Extraction, and post-synthesis simulation. Since the core team has not yet completed all the RTL code, the results presented here are based solely on the register file block.

The `.db` files converted from the `.lib` files are shown below in Fig.3:

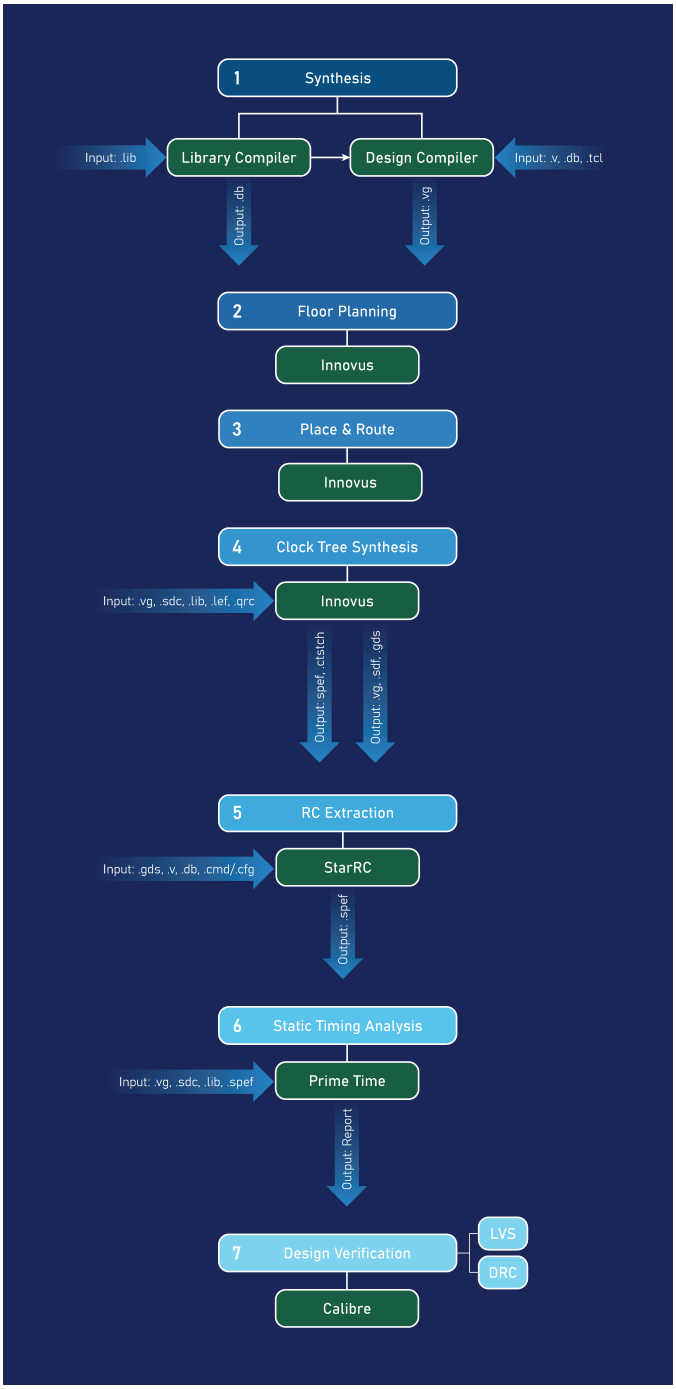

Fig. 2. Physical design flow.

```
[cl228@cobalt library_compiler]$ ls my_lib/
sky130_fd_sc_hd__ff_100C_1v65.db  sky130_fd_sc_hd__ss_n40C_1v28.db
sky130_fd_sc_hd__ff_100C_1v95.db  sky130_fd_sc_hd__ss_n40C_1v35.db
sky130_fd_sc_hd__ff_n40C_1v56.db  sky130_fd_sc_hd__ss_n40C_1v40.db
sky130_fd_sc_hd__ff_n40C_1v65.db  sky130_fd_sc_hd__ss_n40C_1v44.db
sky130_fd_sc_hd__ff_n40C_1v76.db  sky130_fd_sc_hd__ss_n40C_1v60.db
sky130_fd_sc_hd__ff_n40C_1v95.db  sky130_fd_sc_hd__ss_n40C_1v76.db
sky130_fd_sc_hd__ss_100C_1v40.db  sky130_fd_sc_hd__tt_025C_1v80.db
sky130_fd_sc_hd__ss_100C_1v60.db  sky130_fd_sc_hd__tt_100C_1v80.db
```

Fig. 3. List of `.db` files.

The total number of cells used, along with the area, timing, and power results based on the SkyWater PDK, are shown in Fig. 4:

```
Total 4664 cells                                   42176.700007
---------------------------------------------------------------
data required time                                       340.08
data arrival time                                         -3.39
---------------------------------------------------------------
slack (MET)                                              336.69

                  Internal         Switching         Leakage           Total
Power Group       Power            Power             Power             Power   (   %    ) Attrs
--------------------------------------------------------------------------------------------
io_pad            0.0000           0.0000            0.0000            0.0000  (   0.00%)
memory            0.0000           0.0000            0.0000            0.0000  (   0.00%)
black_box         0.0000           0.0000            0.0000            0.0000  (   0.00%)
clock_network     0.1038           0.0000            0.0000            0.1038  (  77.05%)  i
register          2.2105e-04       9.6671e-05        1.5350e+04        1.5668e-02  (  11.63%)
sequential        0.0000           0.0000            0.0000            0.0000  (   0.00%)
combinational     1.8742e-03       5.0947e-03        8.2911e+03        1.5260e-02  (  11.33%)
--------------------------------------------------------------------------------------------
Total             0.1059 mW        5.1914e-03 mW     2.3641e+04 nW     0.1347 mW
```

Fig. 4. Area, timing, and power after synthesis.

Fig. 5 illustrates the latest placement results from the place-and-route stage:

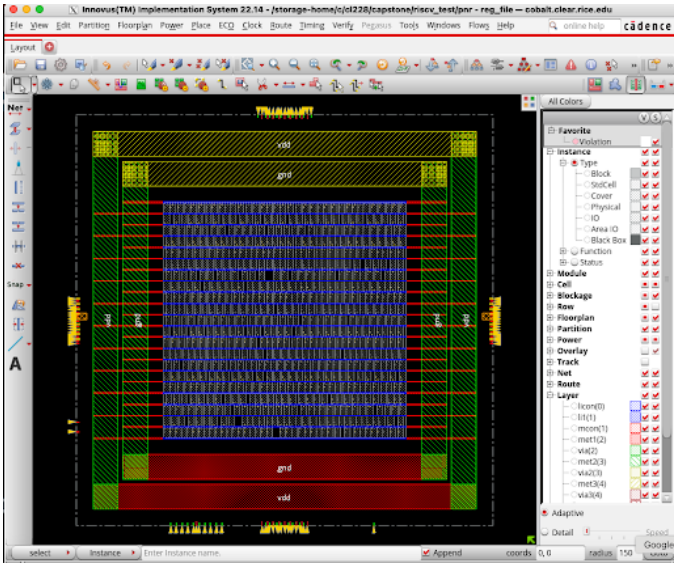

Fig. 5. Placement results from the place-and-route stage.

ACKNOWLEDGMENT

We would like to express our sincerest gratitude to Professor Yu Kee Ooi for this interesting and valuable semester thus far, as well as both Joe Cavallaro and Peter Varman for their resources used by the physical design team. From Joe Cavallaro, the physical design flow framework and files, and from Peter Varman, the regfile.v used in each stage of the methodology.

REFERENCES

[1] Synopsys, "What is RISC-V? – How does it work?," *Synopsys*, https://www.synopsys.com/glossary/what-is-risc-v.html (accessed Mar. 13, 2025).

[2] Rice-MECE-Capstone-Projects, "SWITCHMCU: Switch MCU Repository," *GitHub*, https://github.com/Rice-MECE-Capstone-Projects/SwitchMCU (accessed Mar. 13, 2025).

[3] K. Sharma, “What is Physical Design,” *VLSI - Physical Design For Freshers*, https://www.physicaldesign4u.com/2019/12/what-is-physical-design.html (accessed Mar. 13, 2025).
[4] VLSI Talks, “Floorplanning in Physical Design,” *VLSI TALKS*, https://vlsitalks.com/physical-design/floorplan/ (accessed Mar. 13, 2025).
[5] AnySilicon, “Ultimate Guide: Clock Tree Synthesis,” *AnySilicon*, https://anysilicon.com/clock-tree-synthesis/ (accessed Mar. 13, 2025).
[6] Synopsys, “PrimeTime: Static Timing Analysis,” *Synopsys*, https://www.synopsys.com/implementation-and-signoff/signoff/primetime.html (accessed Mar. 13, 2025).